



## RECOMMENDED LOCATORS

### getByLabel()

RECOMMENDED

Find form inputs by associated <label> text or aria-label.

```
page.getByLabel('Password')
page.getByLabel('Accept terms') // checkbox
page.getByLabel(/username/i) // regex
```

### getByPlaceholder()

RECOMMENDED

Locate input/textarea by its placeholder attribute.

```
page.getByPlaceholder('Search...')
page.getByPlaceholder('Enter your email')
await page.getByPlaceholder('Name').fill('Alice')
```

### getByTestId()

RECOMMENDED

Locate by data-testid attribute — stable across refactors.

```
page.getByTestId('submit-btn')
page.getByTestId('nav-menu')
// Config: use: { testIdAttribute: 'data-qa' }
```

■ Best for test-only IDs that devs won't rename.

### getByRole()

RECOMMENDED

Locate by ARIA role — most resilient, works like a screen reader.

```
page.getByRole('button', { name: 'Submit' })
page.getByRole('textbox', { name: 'Email' })
page.getByRole('heading', { name: /welcome/i })
```

■ Prefer this over CSS/XPath — survives DOM structure changes.

### getByText()

RECOMMENDED

Locate elements by their visible text content.

```
page.getByText('Sign in')
page.getByText('Hello!', { exact: true })
page.getByText(/loading/i) // regex
```

■ Use exact:true to avoid partial matches on long strings.

### getByAltText() / getByTitle()

RECOMMENDED

Locate images by alt text, or any element by title attribute.

```
page.getByAltText('Company logo')
page.getByAltText(/profile photo/i)
page.getByTitle('Close dialog')
```

## CSS & XPATH SELECTORS

### locator('xpath')

CSS/XPATH

XPath 1.0 locator for complex DOM traversal.

```
page.locator('//button[@type="submit"]')
page.locator('//td[text()="Alice"]/../td[2]')
page.locator('xpath=//div[@class="modal"]')
```

■ Use xpath= prefix to be explicit. Avoid absolute XPath paths.

### locator('css selector')

CSS/XPATH

CSS selector — full CSS3 support plus Playwright extensions.

```
page.locator('button.primary')
page.locator('#form input[type="email"]')
page.locator('.card:has(.badge)') // :has() pseudo
```

### :visible / :enabled / :disabled

CSS/XPATH

State pseudo-classes for filtering elements by their state.

```
page.locator('button:visible')
page.locator('input:enabled')
page.locator('input:disabled')
```

### :has-text() / :text-is()

CSS/XPATH

Playwright CSS pseudo-classes for text matching in selectors.

```
page.locator('button:has-text("Save")')
page.locator('.row:has-text("Error")')
page.locator('li:text-is("Home")') // exact
```

## TEXT / FILTER

### filter({ hasText })

TEXT

Narrow a locator set by text content of each matched element.

```
page.locator('.product')
  .filter({ hasText: 'In Stock' })
page.locator('tr').filter({ hasText: /alice/i })
```



## filter({ has })

TEXT

Narrow by a child locator match inside each element.

```
// Rows that contain a checked checkbox
page.locator('tr')
  .filter({ has: page.locator('input:checked') })
```

■ Combines powerfully with `getByRole()` child locators.

## filter({ hasNot })

TEXT

Exclude elements that contain a specific child locator.

```
page.locator('tr')
  .filter({ hasNot: page.locator('.error-icon') })
// Returns rows WITHOUT an error icon
```

## .and() / .or()

CHAINING

Intersect (.and) or union (.or) two locator conditions.

```
page.getByRole('button')
  .and(page.locator(':disabled'))
page.getByRole('button', {name: 'Login'})
  .or(page.getByRole('button', {name: 'Sign in'}))
```

## CHAINING & COMBINING

### .nth(n) / .first() / .last()

CHAINING

Pick element by index from a matched set (zero-indexed).

```
page.getByRole('listitem').nth(0) // first
page.getByRole('listitem').nth(2) // third
page.locator('.notification').last()
```

■ Avoid hardcoded indices — fragile to DOM order changes.

## .locator() child scope

CHAINING

Scope a locator to search within matched parent elements only.

```
page.locator('#billing-form')
  .locator('input[name="city"]')
page.getByRole('dialog')
  .getByRole('button', { name: 'Confirm' })
```

■ Always scope to a parent container for complex pages.

## FRAMES & SHADOW DOM

### frameLocator()

FRAMES

Locate elements inside an `<iframe>` by CSS selector.

```
const frame = page.frameLocator('iframe#editor')
await frame.getByRole('button', {name: 'Bold'}).click()
// Nested iframes:
page.frameLocator('#outer').frameLocator('#inner')
```

## Shadow DOM

FRAMES

Playwright pierces shadow DOM automatically — no special syntax.

```
// Auto-pierces shadow roots transparently
page.locator('my-component button')
page.locator('input').first() // inside shadow too
```

■ No special syntax needed — Playwright handles it automatically.

## BEST PRACTICES

- **getByRole() first**
  - Most resilient — built on ARIA semantics
- **getByTestId() for QA**
  - Stable IDs that survive UI refactors
- **Scope to parent**
  - Scope locators to container for complex pages
- **Avoid .nth()**
  - Index-based locators break on DOM reorder
- **Chain .filter()**
  - Combine `filter()` + `and()` for precision
- **page.pause() to debug**
  - Opens Playwright Inspector for live inspection